

File: seta/1.py

```
j1, j2 = 3, 4
print(j1, j2)

while j1 != 2:
    if j1 == 0:
        j1 = 4
    elif j2 == 3:
        j2 = 0
    else:
        transfer = min(j1, 3 - j2)
        j1 -= transfer
        j2 += transfer
    print(j1, j2)
```

File: seta/2.py

```
board = []
for i in range(9):
    board.append(" ")

player = "X"

for turn in range(9):
    print(board[0:3], "\n", board[3:6], "\n", board[6:9])
    move = int(input("Enter position (0-8): "))
    board[move] = player

    wins = [(0,1,2), (3,4,5), (6,7,8), (0,3,6), (1,4,7), (2,5,8), (0,4,8),
            (2,4,6)]

    win_found = False
    for a, b, c in wins:
        if board[a] == player and board[b] == player and board[c] == player:
            win_found = True
            break

    if win_found:
        print("Player", player, "wins!")
        break

    if player == "X":
        player = "O"
    else:
        player = "X"
```

File: seta/3.py

```
def hanoi(n, a, c, b):
    if n > 0:
        hanoi(n - 1, a, b, c)
        print("Move disk", n, "from", a, "to", c)
        hanoi(n - 1, b, c, a)

n = int(input("Enter number of disks: "))
hanoi(n, 'A', 'C', 'B')
```

File: seta/4.py

```
monkey = "door"
box = "window"
height = "low"
has_banana = False

print("Initial State:", monkey, box, height, has_banana)

while not has_banana:
    if monkey != box:
        print("Action: Monkey moves to", box)
        monkey = box
    elif box != "middle":
        print("Action: Monkey pushes box to middle")
        box = "middle"
        monkey = "middle"
    elif height != "high":
        print("Action: Monkey climbs onto the box")
        height = "high"
    else:
        print("Action: Monkey grabs the banana")
        has_banana = True

print("Final State:", monkey, box, height, has_banana)
```

File: seta/5.py

```
x = 0

while True:
    current_val = -(x - 3)**2
    next_val = -((x + 0.1) - 3)**2

    if next_val > current_val:
        x = x + 0.1
    else:
        break
```

```
print("Optimal x:", round(x, 2))
```

File: setb/1.py

```
def dfs(m, c, b, path):
    if m == 0 and c == 0 and b == 0:
        return path

    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    for dm, dc in moves:
        if b == 1:
            nm = m - dm
            nc = c - dc
        else:
            nm = m + dm
            nc = c + dc

        nb = 1 - b

        if 0 <= nm <= 3 and 0 <= nc <= 3:
            if (nm == 0 or nm >= nc) and (3 - nm == 0 or (3 - nm) >= (3 - nc)):
                if (nm, nc, nb) not in path:
                    new_path = list(path)
                    new_path.append((nm, nc, nb))
                    res = dfs(nm, nc, nb, new_path)
                    if res is not None:
                        return res

    return None

path = dfs(3, 3, 1, [(3, 3, 1)])
for step in path:
    print(step)
```

File: setb/2.py

```
queue = [(3, 3, 1)]

while len(queue) > 0:
    path = queue.pop(0)
    last_state = path[-1]
    m = last_state[0]
    c = last_state[1]
    b = last_state[2]

    if m == 0 and c == 0 and b == 0:
        for step in path:
            print(step)
        break
```

```

moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
for dm, dc in moves:
    if b == 1:
        nm = m - dm
        nc = c - dc
    else:
        nm = m + dm
        nc = c + dc

    nb = 1 - b

    if 0 <= nm <= 3 and 0 <= nc <= 3:
        if (nm == 0 or nm >= nc) and (3 - nm == 0 or (3 - nm) >= (3 - nc)):
            if (nm, nc, nb) not in path:
                new_path = list(path)
                new_path.append((nm, nc, nb))
                queue.append(new_path)

```

File: setb/3.py

```

initial = ["A", "B", "C"]
goal = []

print("Initial:", initial)

while len(initial) > 0:
    block = initial.pop()
    goal.append(block)
    print("Move", block, "to Goal:", goal)

```

File: setb/4.py

```

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def bfs(start, goal):
    q = [start]
    count = 0
    while len(q) > 0:
        count = count + 1
        curr = q.pop(0)
        if curr == goal:

```

```
        return count
    for neighbor in graph[curr]:
        q.append(neighbor)

def dfs(start, goal):
    s = [start]
    count = 0
    while len(s) > 0:
        count = count + 1
        curr = s.pop()
        if curr == goal:
            return count
        for neighbor in graph[curr]:
            s.append(neighbor)

print("BFS visited nodes:", bfs('A', 'F'))
print("DFS visited nodes:", dfs('A', 'F'))
```

File: setc/1.py

```
states = [(0,0), (4,0), (1,3), (1,0), (0,1), (4,1), (2,3)]

for i in range(len(states) - 1):
    current_state = states[i]
    next_state = states[i + 1]
    print(current_state, "to", next_state)
```